Niramoy

Backend rebuild: what was done, and what to do next

27 August 2026 · branch `backend-foundation` · 29 commits · unmerged

29	197	10 / 10	28	4
commits	tests passing	phases complete	database tables	real defects found

> **The one-sentence summary**
>
> The engineering is sound and thoroughly tested — but the AI triage rule set has never been read by a clinician, so **this build is not safe for real patients**, and no further coding changes that.

Your frontend prototype was already calling real API routes; the mocking lived behind the HTTP boundary in two in-memory modules. So this was a backend rebuild, not a rewrite. The design system, layouts and copy are untouched — four files changed, none rewritten.

1 What was built

Phase	Delivered	Proof
1 Foundation	TypeScript, Drizzle ORM, 28-table schema, migrations, seed, typed errors, PHI-redacting logger, health endpoints	21 schema tests
2 Auth	Argon2id, database sessions with rotation, email verification, password reset, CSRF, role-based access	44 security tests
3 Doctors	BM&DC verification workflow, full status lifecycle, admin decisions, suspension	end-to-end verified
4 Scheduling	`end_utc`, exclusion constraint, transactional booking, atomic reschedule, waitlist holds	6 concurrency tests
5 Medical data	Append-only records with amendments, doctor-only prescriptions, consent-gated family access	included above
6 Video	Provider interface (Daily / Jitsi / demo), server-side rooms, scoped expiring tokens	8 authz tests
7 Cron	Five idempotent jobs, authenticated endpoints, reminder and no-show sweeps	11 idempotency tests
8 AI safety	Versioned rules, working Bangla, validated output, urgency floor, draft→review→confirm	55 + 14 tests
9 Payments	Provider interface, labelled mock, signature-verified webhooks, replay protection	13 tests
10 Hardening	Rate limiting, CSP, admin dashboard, CI pipeline, nine documents	CI green

2 The security holes that are now closed

Your prototype resolved the subject of every request as `query.patientId ?? patientIdFor(request)` — preferring a client-supplied id over the session. Adding one query parameter returned anybody's medical records.

#	What it allowed	What closes it
S1	`?patientId=` returned any patient's records, prescriptions and appointments	Subject comes from the session; the parameter is ignored
S2	Signed-out callers were served the demo patient's real data	401
S3	Any caller could issue a prescription, including via a back door on `/api/records`	Doctor role **and** a treatment relationship
S4	Anyone could cancel or complete anyone's appointment by id	Participation checked before every action
S5	Anyone could post any rating against any doctor	Requires an eligible completed appointment
S6	`DELETE ?id=` removed any family member or waitlist entry	Scoped to the owner
S7	Doctor profiles leaked the BM&DC registration number	Public queries list columns explicitly
S8	No CSRF protection on any state-changing route	Origin check + session-bound double-submit token
S9	No verification, reset, rotation, lockout or rate limiting	All implemented
S10	AI endpoints unauthenticated and unbounded	Rate limited, capped, doctor-only where needed

3 Four real defects that writing tests uncovered

01 **Persistent deadlock under concurrent booking**

Twelve simultaneous overlapping inserts deadlocked *stably* — four retries did not clear it, and the burst spent 19 seconds failing. Fixed with a per-doctor advisory lock; the same burst now completes in 113 ms. The layering is deliberate: the exclusion constraint guarantees **correctness**, the lock only buys **liveness**.

02 **Bangla patterns that could never have matched**

The prototype's red-flag rules used `\b` word boundaries, which are defined by ASCII word characters and never match inside Bangla script. Every Bangla term listed was decorative: a patient describing chest pain in Bangla would not have triggered the emergency path at all, and nor would any other Bangla red flag. There is now a test asserting exactly that.

03 **Expected conflicts surfacing as 500s**

Drizzle wraps driver errors, so SQLSTATE and constraint names live on `cause`, not the error you catch. Reading only the top-level message turned a clean 409 into a server error.

04 **Two red-flag coverage gaps**

“face **is** drooping” and “throat **is** closing” did not match patterns written as “face droop” and “throat closing”. Stroke and anaphylaxis, both missed. That is precisely what a vignette suite is for.

4 What is *not* done

Gap	Status	Consequence
Clinical review of the AI rule set	**Blocker**	Triage sensitivity for emergencies is **unmeasured**. Bangladesh's disease burden — dengue, typhoid, TB — is only partly covered.
Legal review (A1–A9)	**Blocker**	Retention, prescribing, cross-border data and payments are all unresolved.
End-to-end browser tests	**Gap**	Playwright is wired and the journeys are named, but no specs exist. Logic is covered at the API level; rendering is not.
Load testing	**Gap**	Scaling figures are reasoned, not measured.

Gap	Status	Consequence
`'unsafe-inline'` in CSP	**Known**	Required by Next's hydration. Removing it means adopting nonces.
Next 15 → 16	**Deferred**	Transitive advisories in `postcss` and `sharp`; a breaking upgrade, out of scope.
MFA	Not built	Doctor and admin accounts are password-only.

5 What you should do next

> **Before anything else — today**
>
> **Merge or review the branch.** 29 commits sit unmerged on `backend-foundation` while `main` is still the vulnerable prototype. Read the commits in order; each explains what was wrong before it. Then `git checkout main && git merge backend-foundation`, or open a PR.

For the CSE-356 submission — highest marks per hour

	Do this	Why it pays
1	**Run the concurrency suite in front of the examiner.** `docker compose up -d` then `npm run test:concurrency`	It is the strongest thing in the project: ten simultaneous bookings, exactly one wins, demonstrated live.
2	**Build the requirements traceability matrix.** Map each proposal requirement to its test in `docs/TESTING.md`	197 tests already exist. This is presentation, not implementation — a few hours for a whole deliverable.
3	**Use the Mermaid diagrams in** `docs/ARCHITECTURE.md` for your report	Request flow, booking, AI triage and verification are already drawn and accurate.
4	**Write the four E2E specs.** Playwright is configured; the journeys are named in the plan	Closes the one unticked box in the Definition of Done.

Before any real patient uses this — non-negotiable

	Do this	Why it matters
1	**Get a doctor to read** `lib/ai/rules.ts`**.** Ask specifically about dengue, typhoid, TB and obstetric emergencies	The largest gap in the project, and the only one engineering cannot close. Everything else is secondary to this.
2	**Build a clinician-labelled vignette set** and measure sensitivity for emergent conditions	Right now the false-negative rate is unknown. "Unknown" is not a safety property.
3	**Take** `docs/REGULATORY_ASSUMPTIONS.md` **to a Bangladeshi lawyer**	A1–A9 are already written as specific questions, so the billable hour is short.
4	**Commission a penetration test**	The security tests cover the attacks I anticipated. A tester finds the ones I did not.

If you keep building

In rough order of value: CSP nonces to remove `'unsafe-inline'`; MFA for doctor and admin accounts; partition `audit_logs` and `safety_events` by month; generate OpenAPI from the existing Zod schemas; move notification delivery to a queue; add a cache for reference data.

6 Practical notes

Topic	What to know
Running it	`npm install && npm run db:migrate && npm run db:seed && npm run dev` — works on a clean checkout with **no environment file**. An in-process PostgreSQL (PGlite) is the default; set `DATABASE_URL` for a real server.
Demo logins	`nabila@` / `ayesha@` / `sakib@example.com`, password `niramoy123`. Admin invite code `NIRAMOY-ADMIN`.
Verifying everything	`npm run verify` runs lint, typecheck, tests and build in order.
Docker is flaky here	The daemon dropped out twice during this session. If the concurrency tests fail with `AggregateError`, that is Docker, not the code — restart it and wait for `pg_isready`.
Production is strict	`APP_ENV=production` makes `DATABASE_URL`, `SESSION_SECRET`, `NIRAMOY_ADMIN_CODE` and `CRON_SECRET` mandatory, refuses defaulted secrets, and refuses the in-process database. Demo profiles are refused too.
Do not scrape BM&DC	The captcha is an access control. Your README already takes this position and the code honours it — if anyone suggests automating it, that is the answer.

Full detail in `docs/FINAL_IMPLEMENTATION_REPORT.md`. Status by category — working, demo-only, needs a provider, needs legal or clinical review, not safe for patients — is in section 11 of that document.